EXACT BILINEAR COMPLEXITY OVER FINITE FIELDS: GUARANTEES FOR A DESCENT HEURISTIC, AND A POOL-FREE SOLVER ROUTE

MOHAMED ALI TEWFIK HAMLIL

Abstract. The rank of a bilinear map is the number of multiplications required to compute it, and deciding it over a finite field is NP-complete. We report on an implementation that attacks the problem from two directions and prove what each one guarantees. For a greedy descent we establish exactness of its first step by Rado–Edmonds, an invariant giving soundness, a termination bound, and a pruning lemma from which the descent's fixed point is shown to be locally optimal against the *entire* candidate pool rather than against the subset it scanned. We prove that the improvement predicate is invariant under any subgroup of the stabiliser of the current span, which is what makes symmetry reduction sound, and observe that the implementation quotients by a subgroup rather than by the full stabiliser, so that enlarging it is a question of efficiency and never of correctness. Separately we note that the satisfiability route requires no enumeration of rank-one maps at any point, so its space is polynomial in the input where the combinatorial route is exponential: on $\langle 4,4,4\rangle$ the pool is 8.2 TiB and refused, while the formula is 11 MB and written in 1.5 s. We also record three negative results, since each cost real time to establish.

1. Introduction

Let $\mathbb{F}$ be a field and let T be a bilinear map, presented as a list of slices $T_1, \dots, T_k$, each an $n \times m$ matrix over $\mathbb{F}$. The *rank* of T, equivalently its *bilinear complexity*, is the least r for which

$$t_{ijl} \;=\; \sum_{s<r} a_i^{(s)}\, b_j^{(s)}\, c_l^{(s)},$$

that is, the least number of multiplications sufficient to compute T. Strassen's seven products in place of eight for 2×2 matrix multiplication is the origin of the subject, and determining rank in general is open.

Two facts frame everything below. First, over a finite field the decision problem "is $\operatorname{rank}(T) \le r$?" is NP-*complete*, not merely NP-hard; the sharper statement is due to Schaefer and Štefankovič, who show that tensor rank over $\mathbb{F}$ is polynomial-time equivalent to the existential theory of $\mathbb{F}$. Membership in NP is not a technicality here: it is what makes a decomposition a certificate of polynomial size, and therefore what makes the problem expressible to a solver at all. Over $\mathbb{R}$ or $\mathbb{Q}$ no such certificate exists, and over $\mathbb{Q}$ the problem is not known to be decidable.

Second, the two natural algorithmic formulations differ in a way that is invisible in their statements and decisive in their space usage. Section 5 makes this precise.

Our contributions are: proofs for a descent heuristic that previously carried measurements and one informal remark (Section 3); the observation that the solver formulation is pool-free and therefore polynomial in space (Section 5); a soundness result for symmetry reduction together with the corollary that closes the gap between it and the implementation (Section 4); and three negative results (Section 7).

Date: August 18, 2026.

2. Preliminaries

Write $\mathrm{cost}(X) = \sum_i \mathrm{rank}(X_i)$ for a list X of matrices. A rank-one bilinear form is exactly one multiplication, so cost counts multiplications.

Definition 2.1. A list S is *feasible* for T if $\mathrm{span}(S) \supseteq \mathrm{span}(T)$.

Feasibility, rather than equality of spans, is the right notion: S need only *generate* T, so a larger spanning set of smaller total rank is a better answer. Karatsuba's five products for a four-coefficient product is exactly such a case, and it is why $|S|$ may exceed $|T|$.

We use throughout that $\mathrm{span}(T) \setminus \{0\}$ under linear independence is a vector matroid, and that $w(M) = \mathrm{rank}(M)$ is a non-negative integer weight on it.

3. A descent, and what it guarantees

The method has three steps. Step 1 chooses a basis of $\mathrm{span}(T)$ of least total rank. Steps 2 and 3 relax feasibility from "basis" to "spanning set" and descend over a pool G of candidate rank-one maps, adopting any candidate whose addition lowers cost. The two later steps are the same procedure differing only in G: step 2 uses the rank-one terms of T itself, step 3 every rank-one map of the shape, one per scalar class, of which there are $(p^n - 1)(p^m - 1)/(p - 1)^2$.

Theorem 3.1 (Step 1 is exact; [7], Theorem 2.1)**.** *Sorting* $\mathrm{span}(T) \setminus \{0\}$ *by ascending rank and keeping each element that preserves independence returns a basis of* $\mathrm{span}(T)$ *of minimum total rank.*

Proof. The procedure is the matroid greedy. By Rado–Edmonds it returns a minimum-weight basis of any matroid under any non-negative weight function. The matroid is the vector matroid of $\mathrm{span}(T)$ and the weight is rank. □

Remark 3.2 (Attribution, and a stronger statement). Theorem 3.1 is not new and is stated here only because everything below depends on it. Nakatsukasa, Soma and Uschmajew pose exactly this minimisation, give exactly this greedy as their Algorithm 1, and make the matroid observation in the same words, calling it a greedy algorithm for an infinite matroid of finite rank [7]. Their Corollary 1 is stronger than the remark below: not only is the optimal *value* independent of tie-breaks, but any other basis of lowest total rank takes the same ranks up to permutation, so the whole multiset is an invariant of $\mathrm{span}(T)$. They also record the caveat that the greedy's oracle, finding a lowest-rank element outside the span so far, is itself NP-hard, so exactness of the *choice* is not tractability of the *problem*. What is new here is the instantiation of that oracle over $\mathbb{F}(p)$ by enumeration, which their nuclear-norm route has no analogue for.

The *value* is therefore independent of how ties are broken, all minimum-weight bases of a matroid having equal weight; the *basis* need not be. Fixing the enumeration order accordingly buys reproducibility, not optimality.

Theorem 3.3 (Soundness)**.** *Every* S *the descent holds is feasible for* T*.*

Proof. By induction. The initial S_0 is a basis of $\mathrm{span}(T)$, so $\mathrm{span}(S_0) = \mathrm{span}(T)$. A step replaces S by a basis of $\mathrm{span}(S \cup \{\varphi\})$, and $\mathrm{span}(S \cup \{\varphi\}) \supseteq \mathrm{span}(S) \supseteq \mathrm{span}(T)$ by hypothesis. □

Consequently a result is never wrong, only possibly large.

Theorem 3.4 (Termination)**.** *The descent performs at most* $\mathrm{cost}(S_0) - \mathrm{rank}(T)$ *adoptions.*

Proof. A candidate is adopted only when cost strictly decreases. By Theorem 3.3 every intermediate S is feasible, so $\mathrm{cost}(S) \geq \mathrm{rank}(T)$. A strictly decreasing sequence of integers bounded below is finite, of length at most the initial gap. □

The next lemma is what licenses the implementation to discard candidates permanently rather than for a single pass, and it is the hypothesis of Theorem 3.6.

Lemma 3.5 (Pruning is sound)**.** *If $\varphi \in \operatorname{span}(S)$ then φ cannot lower* cost*, now or at any later stage of the descent.*

Proof. Each S is a minimum-weight basis of its own span, being the output of Theorem 3.1 applied to that span. If $\varphi \in \operatorname{span}(S)$ then $\operatorname{span}(S \cup \{\varphi\}) = \operatorname{span}(S)$, whose minimum-weight basis has weight $\operatorname{cost}(S)$; there is no decrease. By Theorem 3.3 the span only grows, so $\varphi \in \operatorname{span}(S')$ for every later S' and the argument repeats. □

Theorem 3.6 (Local optimality over the whole pool)**.** *At the fixed point, no $\varphi \in G$ lowers* cost*, including those never tested.*

Proof. Each $\varphi \in G$ was either tested and failed to improve, or discarded, in which case Lemma 3.5 shows it could not have improved. □

Theorem 3.6 is strictly stronger than the loop's stopping condition, which speaks only of the candidates that survived pruning. It is also the claim most exposed to an implementation error, since a defect in the pruning would produce a quietly worse answer rather than a visible failure; it is therefore asserted directly by a test, which rescans the entire pool at the fixed point.

Remark 3.7. No approximation ratio follows. Theorem 3.6 concerns single additions only. A move exchanging two elements at once lies outside the neighbourhood and can pay: $\langle 2, 2, 2\rangle$ sits at a fixed point of cost 8 while its rank is 7, which is why escaping such plateaus is a separate method rather than a parameter of this one.

Remark 3.8. Step 3 cannot do worse than step 2. Step 2's pool consists of rank-one terms of T, each a scalar multiple of an element of step 3's pool, and φ and $c\varphi$ span the same line. The ordering of the steps therefore costs time and not quality.

4. Symmetry reduction

Let $\sigma = (\mu, \nu)$ with μ, ν invertible act on matrices by $M \mapsto \mu^{\mathsf{T}} M \nu$. This is the rank-preserving action under which a decomposition of T is carried to another decomposition.

Theorem 4.1 (Orbit invariance)**.** *If $\sigma(\operatorname{span} S) = \operatorname{span} S$ then φ lowers* cost(S) *if and only if $\sigma(\varphi)$ does.*

Proof. Invertibility of μ, ν gives $\operatorname{rank}(\mu^{\mathsf{T}} M \nu) = \operatorname{rank}(M)$, so σ preserves weights and $\operatorname{cost}(X) = \operatorname{cost}(\sigma(X))$ for every list X. Being a linear isomorphism fixing $\operatorname{span}(S)$, it carries $\operatorname{span}(S \cup \{\varphi\})$ onto $\operatorname{span}(S \cup \{\sigma\varphi\})$. By Theorem 3.1 minimum weight is a function of the span alone, so the two additions have equal cost. □

Hence one representative per orbit of $\operatorname{Stab}(\operatorname{span} S)$ suffices. Note that the hypothesis concerns the stabiliser of the *current* span and not of T: a quotient computed once and reused as S moves is unsound, not merely weaker.

Corollary 4.2. *The conclusion of Theorem 4.1 holds for every subgroup $H \leq \operatorname{Stab}(\operatorname{span} S)$, with orbits at least as fine.*

Proof. The proof of Theorem 4.1 uses only that each element preserves rank and fixes $\operatorname{span}(S)$, which every element of a subgroup does. Finer orbits yield more representatives, hence a smaller reduction and never a wrong answer. □

Corollary 4.2 is what reconciles the theory with the implementation. On the matrix multiplication path the ambient group is supplied as nine generators, and the code retains those that fix the span; the quotient is therefore by the subgroup they generate and not by the full stabiliser. The corollary shows this is sound, and locates the risk in any future improvement: computing a larger subgroup changes performance and cannot change an answer.

Remark 4.3. The action above is not a symmetry a satisfiability solver can exploit internally. Every established SAT symmetry method acts on permutations of literals commuting with negation, whereas σ mixes coordinates linearly. Only the symmetric group on the r summands is expressible in conjunctive normal form. Group-based reduction must therefore enter from outside the solver, as a partition of the search into independent instances, one per orbit of the first term.

5. Two formulations, and why their space differs

The combinatorial formulation searches for a subspace of dimension r possessing a basis of rank-one maps, choosing successive basis elements from an explicit pool. The satisfiability formulation introduces one variable per coordinate of each factor and constrains their products to reproduce T.

The two decide the same question and their space requirements are not comparable. The pool has $(p^n - 1)(p^m - 1)/(p - 1)^2$ elements, exponential in the axis lengths; the formula has $r(n_1 + n_2 + n_3) + r\,n_1n_2 + r\,n_1n_2n_3$ variables, polynomial in the size of T. The rank-one condition in the second is imposed by the shape of the formula rather than by selection from a list, so no enumeration occurs at any point.

$\langle 4, 4, 4\rangle$ at $r = 47$	combinatorial	satisfiability
objects held	4,294,836,225	—
space	8.2 TiB (refused)	11 MB
time to produce	—	1.5 s

Table 1. The same tensor and the same r, both instruments.

The distinction is not incidental to the implementations. It follows from the formulations: "choose the i-th rank-one map" presupposes an enumeration, while "these coordinates multiply out to T" does not.

Remark 5.1. Polynomial space is not tractability. For the combinatorial route a lazily generated pool would change space and not time, converting an immediate failure into a computation that does not terminate in practice. For the solver route the distinction is the whole point, since the search itself is delegated.

Remark 5.2. Membership in FNP is realised literally. On a satisfying assignment the implementation reconstructs the rank-one matrices, multiplies them out, and compares against T, rejecting the answer if they differ. A positive answer therefore does not rest on trusting the solver or the encoding. A negative answer may be accompanied by a DRAT refutation checked by an independent program.

6. Two slices: a theorem that does not survive the field

A tensor with two slices is a matrix pencil $A + xB$, and Kronecker's theory classifies such a pencil up to strict equivalence by its *minimal indices* ε_i, η_j and the *elementary divisors* of its

regular part. Ja'Ja' [4], and independently Grigoriev [5], give the rank from that data: over an algebraically closed field,

$$R(T) \;=\; \sum_{\varepsilon_i>0}(\varepsilon_i+1) \;+\; \sum_{\eta_j>0}(\eta_j+1) \;+\; r \;+\; \delta, \tag{1}$$

where r is the size of the regular part and δ counts the elementary divisors of degree at least two. All of the data in (1) is computable over $\mathrm{GF}(p)$ in polynomial time, which is what `pencil_rank/` does: the singular structure from the ranks of one block system, the divisors from a Smith diagonal over $\mathrm{GF}(p)[x]$ taken twice, forwards for the finite divisors and reversed for those at infinity.

Remark 6.1 (The mistake worth recording). Applying (1) with the elementary divisors computed *over* $\mathrm{GF}(p)$ yields neither side. It is not the rank over the closure, where an irreducible of degree d splits into d distinct linear forms and stops contributing to δ; and it is not the rank over $\mathrm{GF}(p)$. For (I_4, C) over $\mathrm{GF}(2)$ with C the companion matrix of x^4+x+1, that reading gives 5. An exhaustive search proves no decomposition with five products exists, walking 1 897 576 nodes to the end of the tree, and exhibits one with six.

Proposition 6.2. Let T be a two-slice tensor over $\mathrm{GF}(q)$. Then $R_{\mathrm{GF}(q)}(T) \geq R_{\overline{\mathrm{GF}(q)}}(T)$, and the right-hand side is given exactly by (1) with the elementary divisors taken over the closure. If moreover T is diagonalisable over $\mathrm{GF}(q)$, meaning it has no minimal indices and every elementary divisor is a linear form of multiplicity one, the inequality is an equality.

Proof. A decomposition over $\mathrm{GF}(q)$ is a decomposition over any extension, so enlarging the field cannot raise the rank; the value over the closure is (1) by [4]. For the second claim, diagonalisability gives P, Q over $\mathrm{GF}(q)$ with PAQ and PBQ both diagonal, and the r matrices $e_i e_i^{\mathsf{T}}$ are rank one, rational over $\mathrm{GF}(q)$, and span a space containing both. So $R \leq r$, and r is the value of (1) here. □

The gap is a field-size effect rather than a defect in (1), and it is not new. Sumi, Miyazaki and Sakata [6] record the condition exactly. Writing k for the number of invariant polynomials of A that do not factor into *distinct* linear factors over K, their Theorem 3.3, after Ja'Ja', gives $\mathrm{rank}_K(E^n; A) \leq n+k$ *provided* $\mathrm{Card}(K) \geq \deg p_1(A)$; their Theorem 3.5 gives $\mathrm{rank}_K(E^n; A) \geq n+k$ with no hypothesis at all, where Ja'Ja' had the reverse inequality only when $p_1(A)$ splits; and their Proposition 3.4 exhibits (E_3, A) over $\mathrm{GF}(2)$, with A the companion matrix of x^3+x+1, of rank at least 5, so the cardinality hypothesis cannot be dropped.

That proposition is one of the twelve pencils settled by exhaustion here, and this work measured it at exactly 5 against a count of 4 before finding the statement. So the measurements confirm a theorem rather than exhibiting a gap, and `pencil_rank` computes $n+k$ and tests the cardinality condition: where it holds the rank is settled outright, and where it fails $n+k$ remains a proved lower bound over every field. What is left open is the cost *below* the threshold, which is where the three short rows of that table live and where neither theorem applies.

7. Negative results

Each of the following cost time to establish and is recorded so that it is not established again.

Fixed-k search with commitment. A finder that asks only questions it expects to be satisfiable was built on an assumed asymmetry between the cost of acceptance and refutation, quoted at two orders of magnitude. Measured with matched flags, the asymmetry is approximately unity; the original figures compared instances encoded with and without symmetry breaking. The method is dominated by the descent on every fixture measured.

Deeper cube splitting. Partitioning a refutation by orbits of the first two terms rather than the first was measured at 2989.8 s against 108.2 s for the undivided instance, with the longest single part exceeding the whole. Even unbounded parallelism does not recover this.

The expensive step of the descent. Step 3 improved the answer on two of four fixtures, by one product each time, at between 58 and 184 times the cost of the first two steps combined. Effort spent accelerating it is spent on the part that mostly does not pay.

8. Limitations

The descent carries no approximation guarantee, and [8] is why it is unlikely to acquire a good one: approximating the rank of a three-tensor within $1 + 1/1852$ is NP-hard *over any field*, so the obstruction is not an artefact of working over $\mathbb{F}(p)$ and it bounds every method here at once. The exhaustive route is complete but its cost is $\binom{|G|}{r-\dim T}$, which is prohibitive beyond small shapes. Reported timings are machine-dependent and are not reproducible in continuous integration; counts are, and are asserted there. Nothing here settles the bilinear rank problem.

References

[1] M. Schaefer and D. Štefankovič, *The complexity of tensor rank*, Theory of Computing Systems, 2018.
[2] J. Håstad, *Tensor rank is NP-complete*, Journal of Algorithms 11 (1990), 644–654.
[3] R. Barbulescu, J. Detrey, N. Estibals and P. Zimmermann, *Finding optimal formulae for bilinear maps*, WAIFI 2012.
[4] J. Ja'Ja', *Optimal evaluation of pairs of bilinear forms*, SIAM Journal on Computing 8 (1979), 443–462.
[5] D. Yu. Grigoriev, *Some new bounds on tensor rank*, LOMI preprint, 1978.
[6] T. Sumi, M. Miyazaki, T. Sakata, *Rank of 3-tensors with 2 slices and Kronecker canonical forms*, Linear Algebra and its Applications **431** (2009), 1858–1868.
[7] Y. Nakatsukasa, T. Soma, A. Uschmajew, *Finding a low-rank basis in a matrix subspace*, Mathematical Programming **162** (2017), 325–361.
[8] J. Swernofsky, *Tensor rank is hard to approximate*, APPROX/RANDOM 2018, LIPIcs **116**, 26:1–26:9. (Factor $1 + 1/1852$, over any field.)
[9] J. Oxley, *Matroid Theory*, 2nd ed., Oxford, 2011. (Rado–Edmonds, the matroid greedy.)
[10] B. D. McKay, *Isomorph-free exhaustive generation*, Journal of Algorithms 26 (1998), 306–324.
[11] The CASYS team, *Givaro: exact arithmetic over $\mathbb{F}(p)$ and over the rationals.*